안녕하세요.
Javascript 강좌중인 Falcon 입니다.

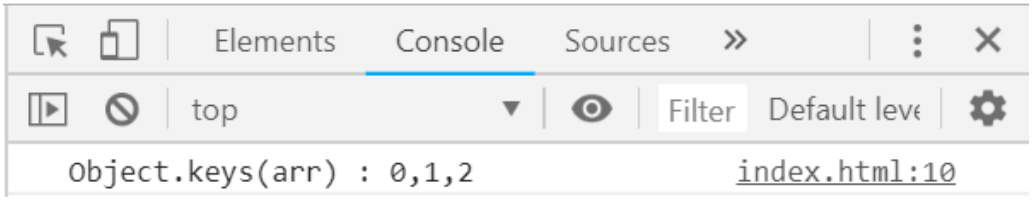
Object 객체는 객체의 가장 기본적인 형태를 가지고 있는 객체이다. 다시 말해서 아무것도 상속받지 않는 순수한 객체다. 자바스크립트에서는 값을 저장하는 기본적인 단위로 Object를 사용한다. 동시에 자바스크립트의 모든 객체는 Object객체를 상속 받는데, 그런 이유로 모든 객체는 Object 객체의 프로퍼티를 가지고 있다.

```
8   <script type="text/javascript">
9     var arr = {'a':1, 'b':2,'c':3 };
10  </script>
```

여러분들 눈으로 보이는 것은 arr밖에 없지만 이 객체 위에는 암시적으로 Object라는 객체가 존재하고 있습니다.

```
8   <script type="text/javascript">
9     var arr = ['a', 'b', 'c'];
10    console.log('Object.keys(arr) : ' + Object.keys(arr));
11  </script>
```

Object객체 내에는 여러분들이 모르고 있는 메소드들이 많습니다. 그 중에서 keys라는 메소드를 사용해보도록 하겠습니다. 그런데 저는 여러분들에게 keys의 기능을 알려주려는 것이 아닙니다. keys 메소드는 prototype 상속으로 선언된 메소드가 아니기 때문에 인자 값을 올려줘야한다는 것 입니다.



그리고 이 keys의 기능은 arr 배열의 인덱스 번호를 출력해주는 기능입니다. 쉽게 말하면 객체의 속성값을 알려주는 메소드 입니다. 그런데 Object안의 메소드 중에서는 prototype으로 선언된 메소드들이 있습니다.

```
8   <script type="text/javascript">
9     var o = new Array(1,2,3);
10    console.log('o.toString() : ' + o.toString());
11  </script>
```

바로 toString는 prototype의 메소드입니다. 이런 부분들은 Javascript reference를 보시면 toString가 prototype인것을 볼 수 있으실 겁니다. 아래는 Object가 쓸 수 있는 메소드의 일부분입니다.

Object.prototype.eval() [Translate]

Object.prototype.hasOwnProperty()

Object.prototype.isPrototypeOf()

Object.prototype.property
IsEnumerable()

Object.prototype.toLocaleString()

Object.prototype.toSource() [Translate]

Object.prototype.toString()

Object.prototype.unwatch() [Translate]

Object.prototype.valueOf()

Object.prototype.watch() [Translate]

Object.seal()

Object.setPrototypeOf()

Object.unobserve() [Translate]

Object.values()

여러분들에게 눈으로 확인 시켜드리기 위해서 직접 사이트에 들어가서 찍어온 스샷 입니다.

```
8        <script type="text/javascript">
9          var o = new Array(1,2,3);
10         console.log('o.toString() : ' + o.toString());
11       </script>
```

prototype로 되어 있다는 것은 이 메소드를 다른 객체에서도 쓸 수 있다는 것을 의미 합니다. 그렇기 때문에 keys와는 달리 인자값을 넘겨주지 않아도 o.toString()를 통해서 결과를 확인 할 수 있습니다.

Elements Console Sources »

top Filter Default leve

```
o.toString() : 1,2,3                    index.html:10
```

위의 코드를 직접 실행을 해보시면 결과가 정상적으로 출력되는 것을 볼 수 있습니다. toString가 하는 기능은 배열 안에 담긴 데이터를 읽어들이는 기능을 가지고 있습니다. 9행에서 1,2,3을 출력한 것을 볼 수 있습니다.

```
8   <script type="text/javascript">
9     var o = {'name' : 'Falcon', 'city' : 'seoul'}
10    console.log(o.contain('Falcon'));
11    var a = ['Falcon','Spider','devi'];
12    console.log(a.contain('Spider'));
13  </script>
```

위의 코드를 실행을 위한 코드가 아닙니다. 지금부터 여러분들에게 Object를 이용해서 내가 만든 객체에 기능을 추가해보려고 합니다. 그 기능은 9행은 객체로 만들었고, 10행은 배열로 만들었는데 여기에 내가 원하는 이름이 존재한다면 true를 반환해주고, 만약 존재하지 않는다면 false를 반환하는 기능을 만들어 볼겁니다.

```
8   <script type="text/javascript">
9     Object.prototype.contain = function(neddle) {
10      for(var name in this) {
11        if(this[name] === neddle) {
12          return true;
13        }
14      }
15      return false;
16    }
17    var o = {'name' : 'Falcon', 'city' : 'seoul'}
18    console.log(o.contain('Falcon'));
19    var a = ['Falcon','Spider','devil'];
20    console.log(a.contain('Spider'));
21  </script>
```

방법은 간단합니다. 다만 여러분들이 상속에 대한 이해와 prototype에 대한 이해를 가지고 계셔야합니다. 저희가 상속과 prototype에 대해 배웠듯이 Object는 상위객체이며 어떠한 객체든 이 Object를 상속하고 있기 때문에 prototype을 이용해서 모든 객체들에게 함수를 뿌려줄 수 있습니다. 그런데 위의 방식은 위험합니다. 특히 협업시에 의논을 하지 않은 상태라면 더더욱이 위험하게 됩니다. 그럼 무엇이 위험한지 알아봅시다.

```
8        <script type="text/javascript">
9          Object.prototype.contain = function(neddle) {
10           for(var name in this) {
11             if(this[name] === neddle) {
12               return true;
13             }
14           }
15           return false;
16         }
17         var o = {'name' : 'Falcon', 'city' : 'seoul'}
18         console.log(o.contain('Falcon'));
19         var a = ['Falcon','Spider','devil'];
20         console.log(a.contain('Spider'));
21         var b = ['rabbit','dog','cat'];
22         for(var i in b) {
23           console.log(b[i]);
24         }
25       </script>
```

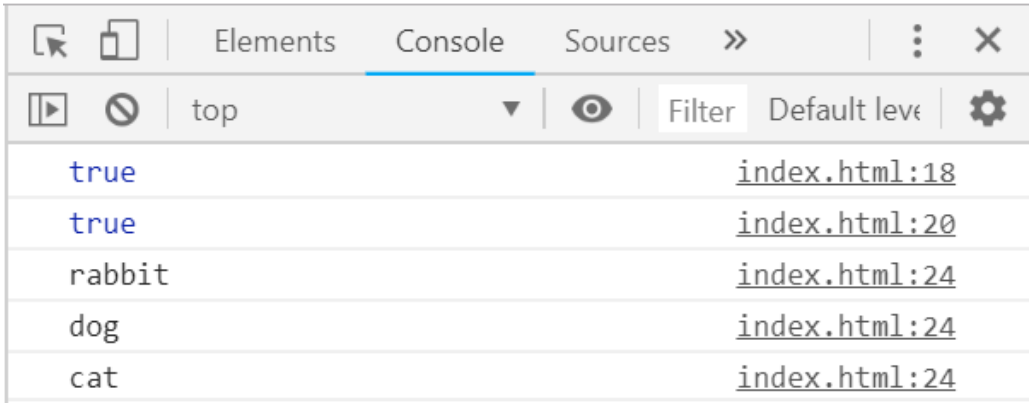
21행 ~ 24행의 코드는 여러분들 개인이 아니 같이 일하는 작업자라고 생각을 해봅시다. 이사람은 단순히 console.log를 통해서 배열 b가가지고 있는 값을 출력하려고 했습니다. 여러분들이 만든 18행과 20행은 하나의 출력이기 때문에 문제가 생기지 않지만 다든 사람이 만든 코드에는 문제가 하나 생기게 됩니다.

```
Elements   Console   Sources   »
top ▼   Filter   Default leve
true                                   index.html:18
true                                   index.html:20
rabbit                                 index.html:23
dog                                    index.html:23
cat                                    index.html:23
ƒ (neddle) {                           index.html:23
        for(var name in this) {
          if(this[name] === neddle) {
            return true;
          }
        }
        return false;
      }
```

여러분들이 만든 18행과 20행 코드는 정상적으로 실행되지만 23행 부분을 보면 함수가 그대로 드러나는 것을 볼 수 있습니다. 물론 이 부분이 뜨지 않게 하는 방법은 있지만 의논하지 않은 채 Object를 이용해서 기능을 구현할 경우 이러한 문제점이 일어나게 됩니다. 함수의 코드가 그대로 드러나게 된다는 겁니다.

```
8         <script type="text/javascript">
9           Object.prototype.contain = function(neddle) {
10            for(var name in this) {
11              if(this[name] === neddle) {
12                return true;
13              }
14            }
15            return false;
16          }
17          var o = {'name' : 'Falcon', 'city' : 'seoul'}
18          console.log(o.contain('Falcon'));
19          var a = ['Falcon','Spider','devil'];
20          console.log(a.contain('Spider'));
21          var b = ['rabbit','dog','cat'];
22          for(var i in b) {
23            if(b.hasOwnProperty(i)) {
24              console.log(b[i]);
25            }
26          }
27        </script>
```

해결방법은 hasOwnProperty() 메소드를 통해서 해결이 가능합니다. 이렇게 선언을 하면 해결은 가능하지만 협업시 의논하지 않고 Object 객체에 추가를 할 경우 이러한 문제점이 있다는 점 기억을 해주시길 바라며 웬만하면 Object를 이용한 방법은 개인 프로젝트에서만 사용하시길 추천드립니다.



hasOwnProperty() 메소드를 이용해서 실행을 해보면 Object에서 추가한 기능이 나오지 않는 것을 확인 할 수 있습니다. hasOwnProperty의 기능은 원래가지고 있는 원소들만 출력하는 기능 입니다.

이것으로 [Javascript 강좌] 제 27강 Object 편을
마치도록 하겠습니다.